```
# Import required libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files

# Display settings
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 100)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
# Upload your dataset
uploaded = files.upload()

filename = list(uploaded.keys())[0]
print("Uploaded file:", filename)

# Read CSV file
df = pd.read_csv(filename)

print("Dataset loaded successfully!")
print("Shape:", df.shape)
```

Choose Files superstore.csv.zip
superstore.csv.zip(application/x-zip-compressed) - 3349507 bytes, last modified: 9/22/2026 - 100% done

```
Saving superstore.csv.zip to superstore.csv.zip
Uploaded file: superstore.csv.zip
Dataset loaded successfully!
Shape: (51290, 27)
```

```
# Display first 10 records
df.head(10)
```

	Category	City	Country	Customer.ID	Customer.Name	Discount	Market	记录数	Order.Date	Order.ID	Order.Priority	Product.ID
0	Office Supplies	Los Angeles	United States	LS-172304	Lycoris Saunders	0.0	US	1	2011-01-07 00:00:00.000	CA-2011-130813	High	OFF-PA-10002005
1	Office Supplies	Los Angeles	United States	MV-174854	Mark Van Huff	0.0	US	1	2011-01-21 00:00:00.000	CA-2011-148614	Medium	OFF-PA-10002893
2	Office Supplies	Los Angeles	United States	CS-121304	Chad Sievert	0.0	US	1	2011-08-05 00:00:00.000	CA-2011-118962	Medium	OFF-PA-10000659
3	Office Supplies	Los Angeles	United States	CS-121304	Chad Sievert	0.0	US	1	2011-08-05 00:00:00.000	CA-2011-118962	Medium	OFF-PA-10001144
4	Office Supplies	Los Angeles	United States	AP-109154	Arthur Prichep	0.0	US	1	2011-09-29 00:00:00.000	CA-2011-146969	High	OFF-PA-10002105
5	Office Supplies	Los Angeles	United States	JF-154904	Jeremy Farry	0.0	US	1	2011-10-19 00:00:00.000	CA-2011-117317	Critical	OFF-PA-10004519
6	Office Supplies	Los Angeles	United States	WB-218504	William Brown	0.0	US	1	2011-11-04 00:00:00.000	CA-2011-125829	Medium	OFF-PA-10000223
7	Office Supplies	Los Angeles	United States	JA-159704	Joseph Airdo	0.0	US	1	2011-11-12 00:00:00.000	CA-2011-151295	High	OFF-PA-10001947
8	Office Supplies	Los Angeles	United States	SP-209204	Susan Pistek	0.0	US	1	2011-11-22 00:00:00.000	CA-2011-135090	High	OFF-PA-10002245
9	Office Supplies	Los Angeles	United States	RL-196154	Rob Lucas	0.0	US	1	2011-12-05 00:00:00.000	CA-2011-133830	Medium	OFF-PA-10001363

```
# Dataset information

print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

print("\nColumn Names:")
print(df.columns.tolist())

print("\nDataset Information:")
df.info()
```

```
Number of rows: 51290
Number of columns: 27

Column Names:
['Category', 'City', 'Country', 'Customer.ID', 'Customer.Name', 'Discount', 'Market', '记录数', 'Order.Date', 'Order.ID', 'Order

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51290 entries, 0 to 51289
Data columns (total 27 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Category        51290 non-null  object
 1   City            51290 non-null  object
 2   Country         51290 non-null  object
 3   Customer.ID     51290 non-null  object
 4   Customer.Name   51290 non-null  object
 5   Discount        51290 non-null  float64
 6   Market          51290 non-null  object
 7   记录数             51290 non-null  int64
 8   Order.Date      51290 non-null  object
 9   Order.ID        51290 non-null  object
 10  Order.Priority  51290 non-null  object
 11  Product.ID      51290 non-null  object
 12  Product.Name    51290 non-null  object
 13  Profit          51290 non-null  float64
 14  Quantity        51290 non-null  int64
 15  Region          51290 non-null  object
 16  Row.ID          51290 non-null  int64
```

```
 17  Sales           51290 non-null  int64
 18  Segment         51290 non-null  object
 19  Ship.Date       51290 non-null  object
 20  Ship.Mode       51290 non-null  object
 21  Shipping.Cost   51290 non-null  float64
 22  State           51290 non-null  object
 23  Sub.Category    51290 non-null  object
 24  Year            51290 non-null  int64
 25  Market2         51290 non-null  object
 26  weeknum         51290 non-null  int64
dtypes: float64(3), int64(6), object(18)
memory usage: 10.6+ MB
```

```
# Check missing values

missing = df.isnull().sum()

print("Missing values in each column:")
print(missing)

print("\nTotal missing values:", df.isnull().sum().sum())
```

```
Missing values in each column:
Category          0
City              0
Country           0
Customer.ID       0
Customer.Name     0
Discount          0
Market            0
记录数               0
Order.Date        0
Order.ID          0
Order.Priority    0
Product.ID        0
Product.Name      0
Profit            0
Quantity          0
Region            0
Row.ID            0
Sales             0
Segment           0
Ship.Date         0
Ship.Mode         0
Shipping.Cost     0
State             0
Sub.Category      0
Year              0
Market2           0
weeknum           0
dtype: int64

Total missing values: 0
```

```
# Check duplicate records

duplicates = df.duplicated().sum()

print("Number of duplicate records:", duplicates)

# Remove duplicates
df = df.drop_duplicates()

print("Dataset shape after removing duplicates:", df.shape)
```

```
Number of duplicate records: 0
Dataset shape after removing duplicates: (51290, 27)
```

```
# Statistical summary

df.describe(include='all').T
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
Category	51290	3	Office Supplies	31273	NaN	NaN	NaN	NaN	NaN	NaN	NaN
City	51290	3636	New York City	915	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Country	51290	147	United States	9994	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Customer.ID	51290	4873	JG-158051	40	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Customer.Name	51290	795	Muhammed Yedwab	108	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Discount	51290.0	NaN	NaN	NaN	0.142908	0.21228	0.0	0.0	0.0	0.2	0.85
Market	51290	7	APAC	11002	NaN	NaN	NaN	NaN	NaN	NaN	NaN
记录数	51290.0	NaN	NaN	NaN	1.0	0.0	1.0	1.0	1.0	1.0	1.0
Order.Date	51290	1430	2014-06-18 00:00:00.000	135	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Order.ID	51290	25035	CA-2014-100111	14	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Order.Priority	51290	4	Medium	29433	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product.ID	51290	10292	OFF-AR-10003651	35	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product.Name	51290	3788	Staples	227	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Profit	51290.0	NaN	NaN	NaN	28.610982	174.340972	-6599.978	0.0	9.24	36.81	8399.976
Quantity	51290.0	NaN	NaN	NaN	3.476545	2.278766	1.0	2.0	3.0	5.0	14.0
Region	51290	13	Central	11117	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Row.ID	51290.0	NaN	NaN	NaN	25645.5	14806.29199	1.0	12823.25	25645.5	38467.75	51290.0
Sales	51290.0	NaN	NaN	NaN	246.49844	487.567175	0.0	31.0	85.0	251.0	22638.0
Segment	51290	3	Consumer	26518	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Ship.Date	51290	1464	2014-11-22 00:00:00.000	130	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Ship.Mode	51290	4	Standard Class	30775	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Shipping.Cost	51290.0	NaN	NaN	NaN	26.375818	57.29681	0.002	2.61	7.79	24.45	933.57

```
# Convert possible date columns into datetime format

for col in df.columns:
    if 'date' in col.lower():
        try:
            df[col] = pd.to_datetime(df[col], errors='coerce')
            print(f"Converted to date: {col}")
        except:
            pass
```

```
Converted to date: Order.Date
Converted to date: Ship.Date
```

```
# Display all columns for identifying business metrics

for i, col in enumerate(df.columns, 1):
    print(i, ":", col)
```

```
1 : Category
2 : City
3 : Country
4 : Customer.ID
5 : Customer.Name
6 : Discount
7 : Market
8 : 记录数
9 : Order.Date
10 : Order.ID
11 : Order.Priority
```

```
12 : Product.ID
13 : Product.Name
14 : Profit
15 : Quantity
16 : Region
17 : Row.ID
18 : Sales
19 : Segment
20 : Ship.Date
21 : Ship.Mode
22 : Shipping.Cost
23 : State
24 : Sub.Category
25 : Year
26 : Market2
27 : weeknum
```

```
# Automatically find common business columns

def find_column(possible_names):
    for name in possible_names:
        for col in df.columns:
            if col.lower().strip() == name.lower():
                return col
    return None

sales_col = find_column(['Sales', 'Revenue', 'Amount'])
profit_col = find_column(['Profit', 'Net Profit'])
quantity_col = find_column(['Quantity', 'Qty'])

print("Sales column:", sales_col)
print("Profit column:", profit_col)
print("Quantity column:", quantity_col)
```

```
Sales column: Sales
Profit column: Profit
Quantity column: Quantity
```

```
# Calculate KPIs

if sales_col:
    total_sales = df[sales_col].sum()
else:
    total_sales = 0

if profit_col:
    total_profit = df[profit_col].sum()
else:
    total_profit = 0

if quantity_col:
    total_quantity = df[quantity_col].sum()
else:
    total_quantity = 0

profit_margin = (total_profit / total_sales) * 100 if total_sales != 0 else 0

print("=" * 45)
print("             BUSINESS KPI REPORT")
print("=" * 45)

print(f"Total Sales     : {total_sales:,.2f}")
print(f"Total Profit    : {total_profit:,.2f}")
print(f"Total Quantity  : {total_quantity:,.0f}")
print(f"Profit Margin   : {profit_margin:.2f}%")

print("=" * 45)
```

```
=============================================
             BUSINESS KPI REPORT
=============================================
Total Sales     : 12,642,905.00
Total Profit    : 1,467,457.29
Total Quantity  : 178,312
Profit Margin   : 11.61%
=============================================
```

```
# Correlation matrix for numerical columns
```

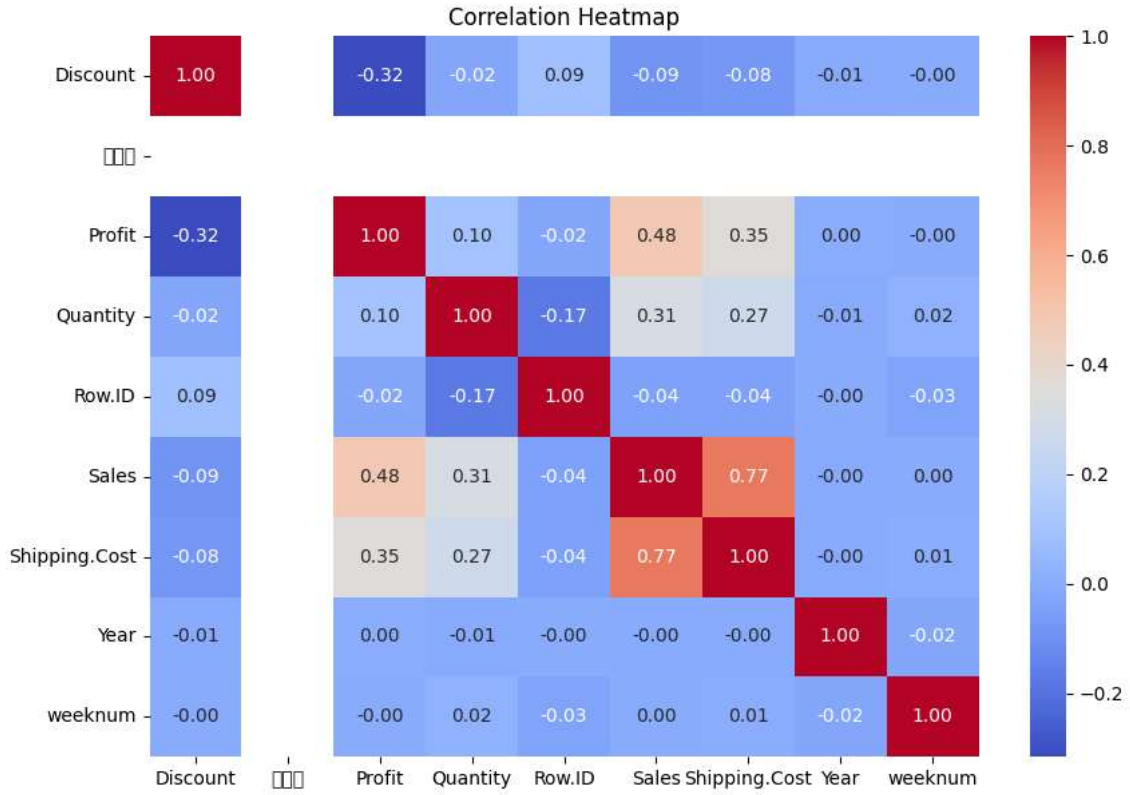
```
numeric_df = df.select_dtypes(include=np.number)

plt.figure(figsize=(10, 7))

sns.heatmap(
    numeric_df.corr(),
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title("Correlation Heatmap")
plt.show()
```

```
/usr/local/lib/python3.13/dist-packages/seaborn/utils.py:61: UserWarning: Glyph 35760 (\N{CJK UNIFIED IDEOGRAPH-8BB0}) missing f
  fig.canvas.draw()
/usr/local/lib/python3.13/dist-packages/seaborn/utils.py:61: UserWarning: Glyph 24405 (\N{CJK UNIFIED IDEOGRAPH-5F55}) missing f
  fig.canvas.draw()
/usr/local/lib/python3.13/dist-packages/seaborn/utils.py:61: UserWarning: Glyph 25968 (\N{CJK UNIFIED IDEOGRAPH-6570}) missing f
  fig.canvas.draw()
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 35760 (\N{CJK UNIFIED IDEOGRAPH-8BB0}
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 24405 (\N{CJK UNIFIED IDEOGRAPH-5F55}
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 25968 (\N{CJK UNIFIED IDEOGRAPH-6570}
  fig.canvas.print_figure(bytes_io, **kw)
```

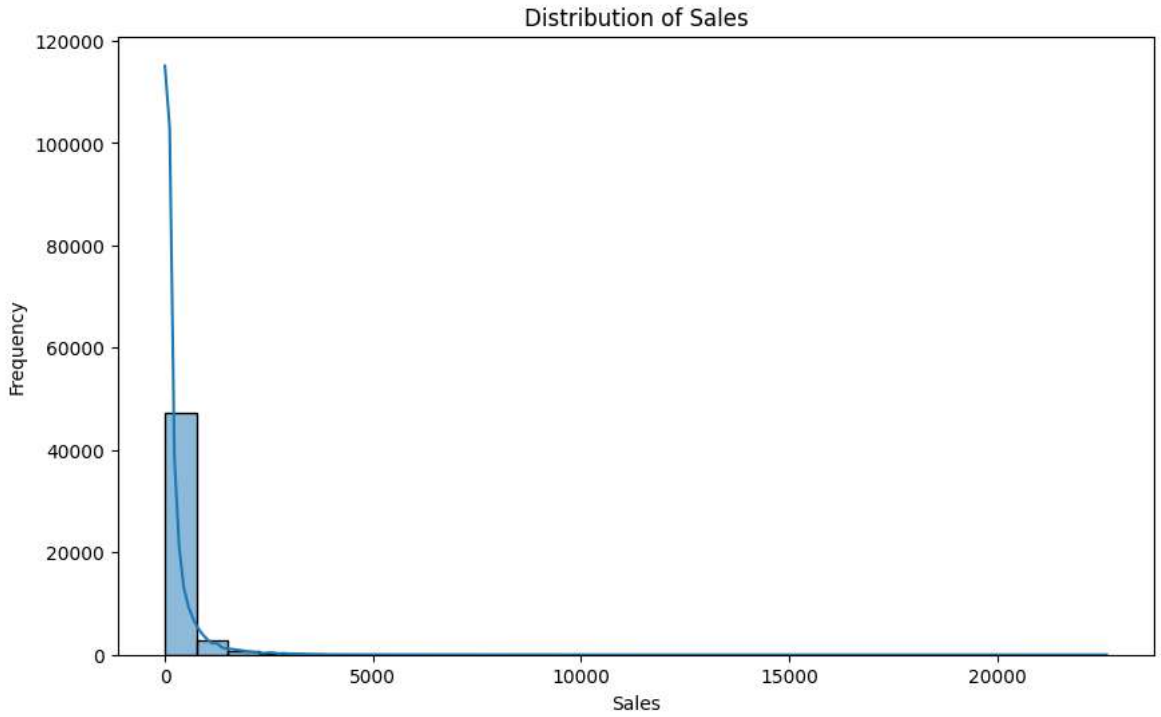



```
if sales_col:
    plt.figure(figsize=(10, 6))

    sns.histplot(df[sales_col], bins=30, kde=True)

    plt.title("Distribution of Sales")
    plt.xlabel("Sales")
    plt.ylabel("Frequency")

    plt.show()
```

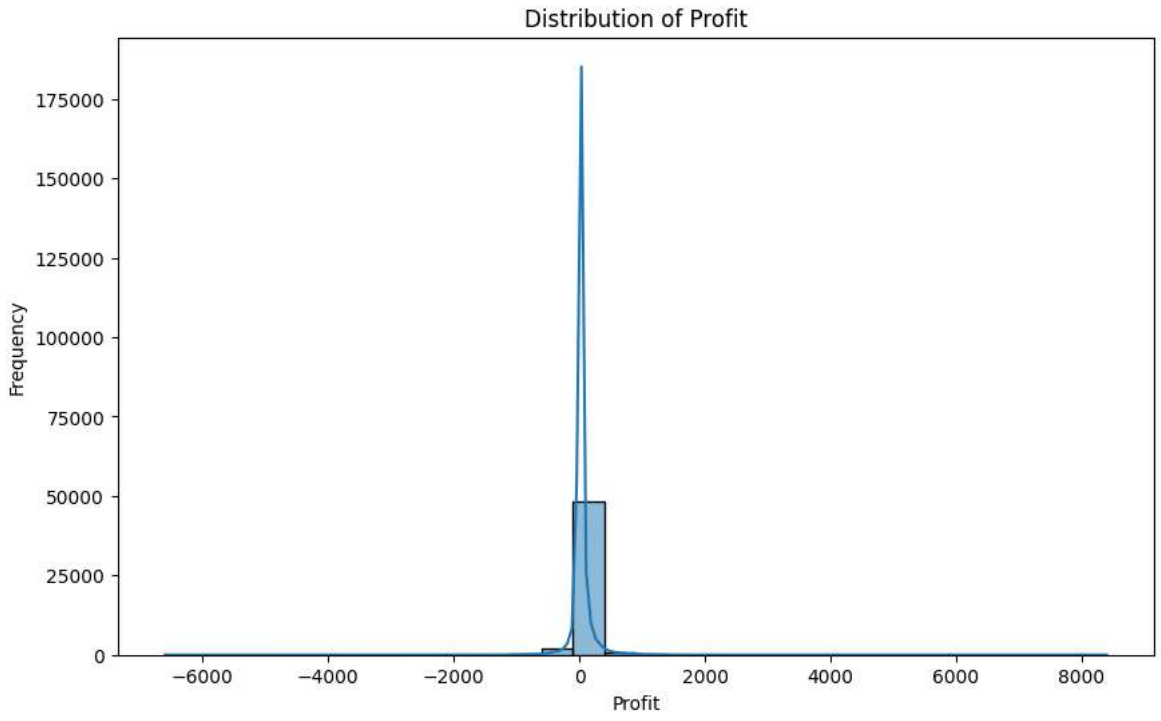



```
if profit_col:
    plt.figure(figsize=(10, 6))

    sns.histplot(df[profit_col], bins=30, kde=True)

    plt.title("Distribution of Profit")
    plt.xlabel("Profit")
    plt.ylabel("Frequency")

    plt.show()
```




```
category_col = None

for col in df.columns:
    if col.lower() == 'category':
        category_col = col
```

```
        break

if category_col and sales_col:

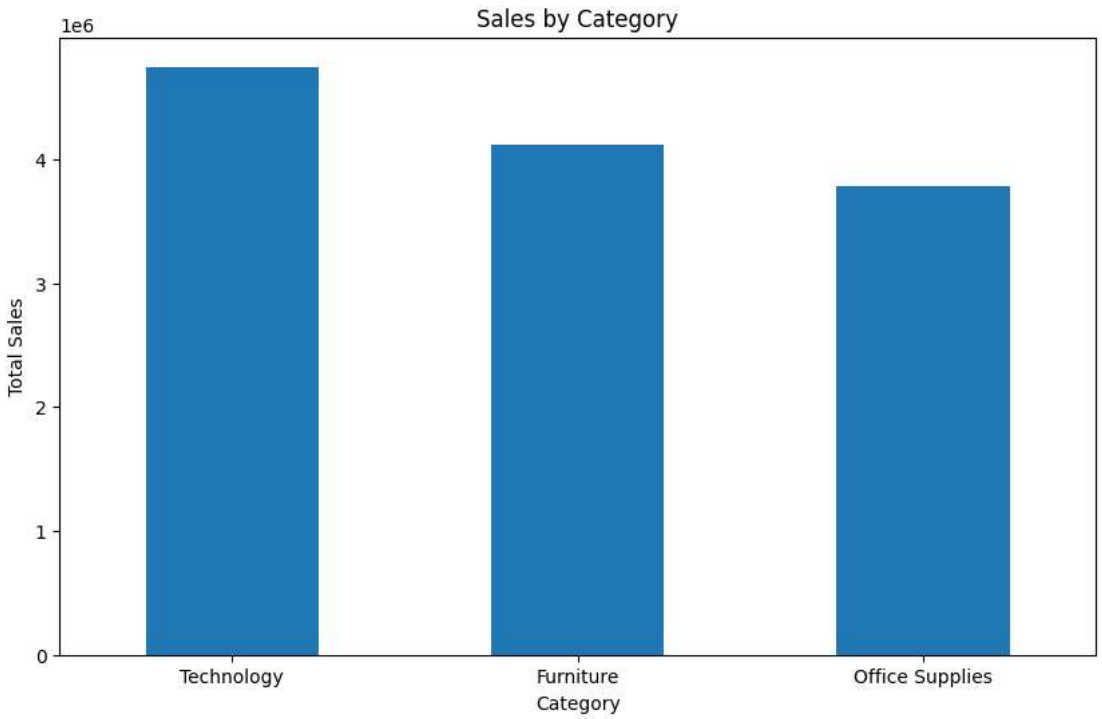
    category_sales = df.groupby(category_col)[sales_col].sum().sort_values(ascending=False)

    plt.figure(figsize=(10, 6))

    category_sales.plot(kind='bar')

    plt.title("Sales by Category")
    plt.xlabel("Category")
    plt.ylabel("Total Sales")

    plt.xticks(rotation=0)
    plt.show()
```




```
if category_col and profit_col:

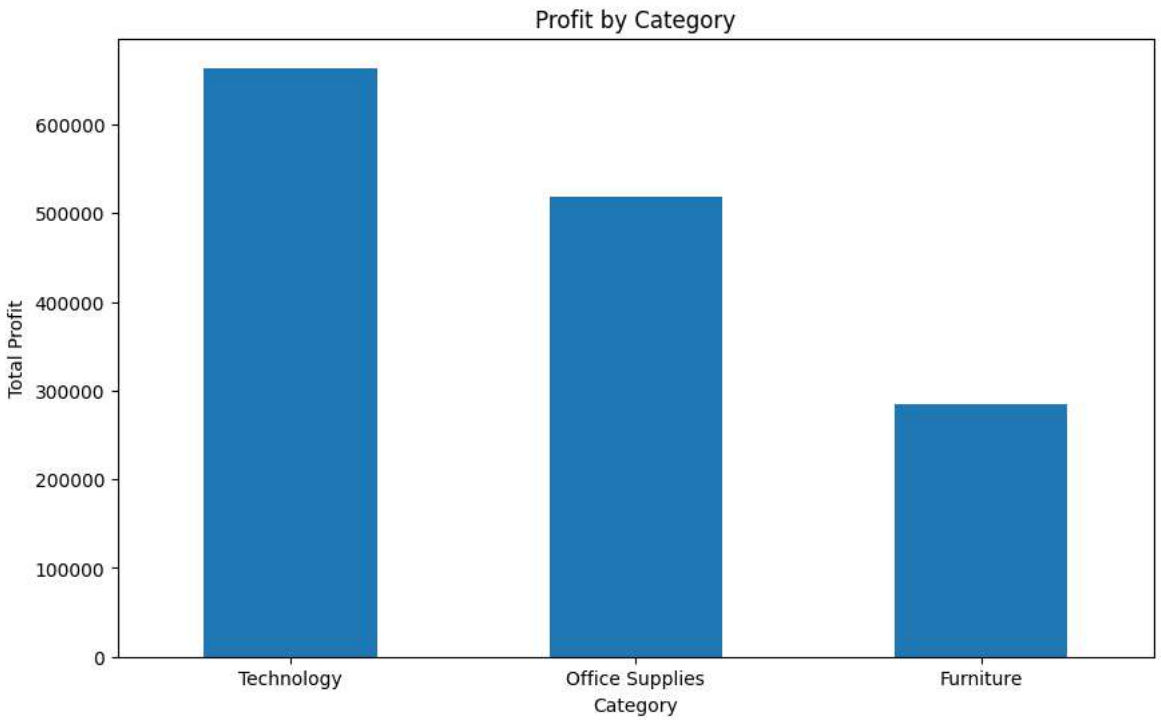
    category_profit = df.groupby(category_col)[profit_col].sum().sort_values(ascending=False)

    plt.figure(figsize=(10, 6))

    category_profit.plot(kind='bar')

    plt.title("Profit by Category")
    plt.xlabel("Category")
    plt.ylabel("Total Profit")

    plt.xticks(rotation=0)
    plt.show()
```




```
region_col = None

for col in df.columns:
    if col.lower() == 'region':
        region_col = col
        break

if region_col and sales_col:

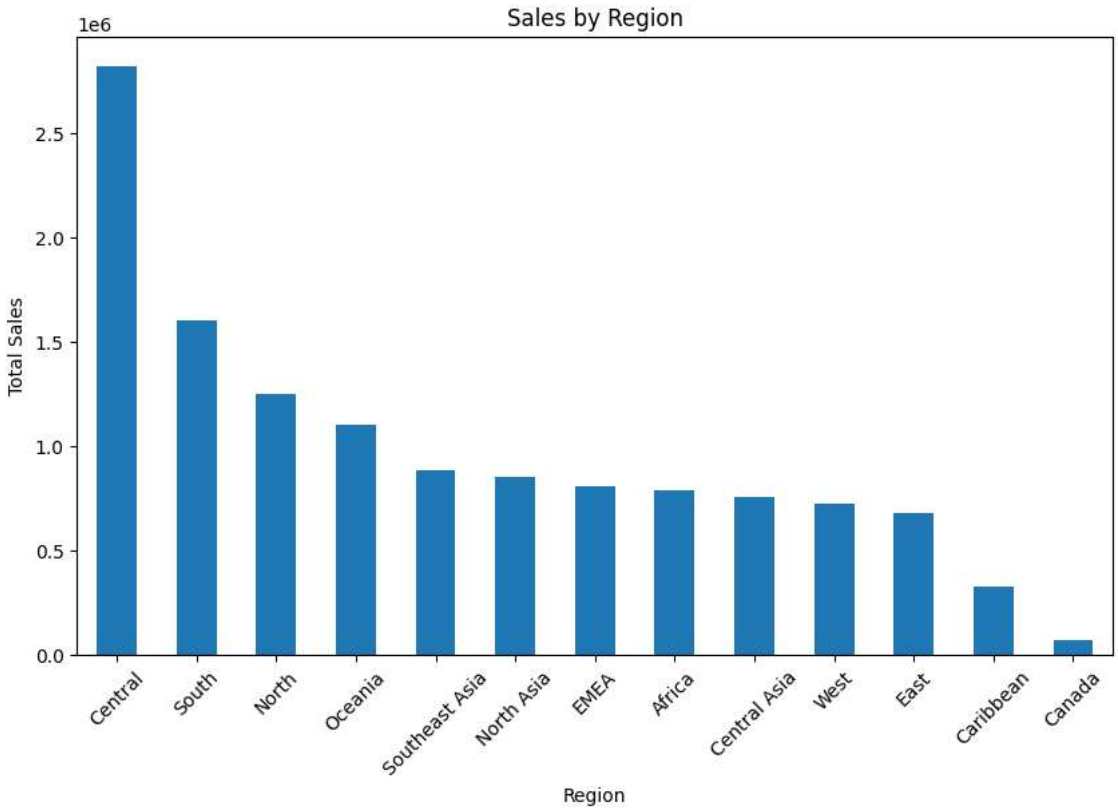
    region_sales = df.groupby(region_col)[sales_col].sum().sort_values(ascending=False)

    plt.figure(figsize=(10, 6))

    region_sales.plot(kind='bar')

    plt.title("Sales by Region")
    plt.xlabel("Region")
    plt.ylabel("Total Sales")

    plt.xticks(rotation=45)
    plt.show()
```




```
if region_col and profit_col:

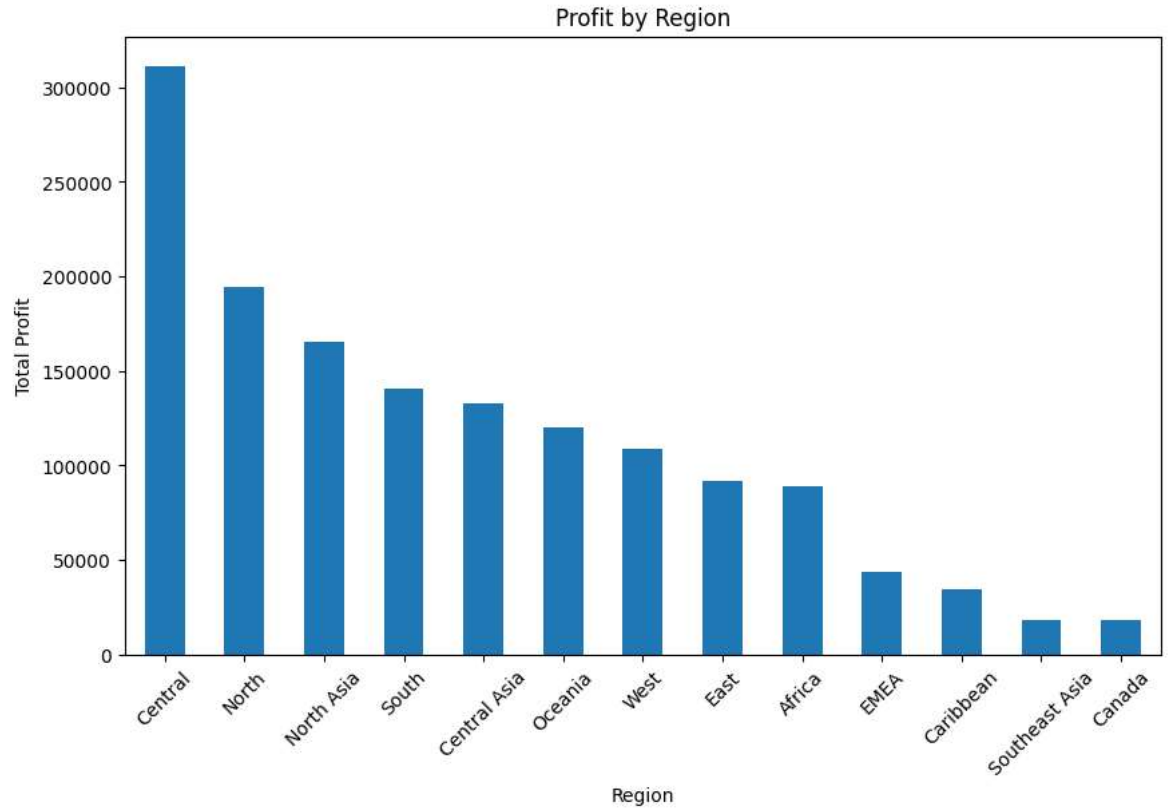
    region_profit = df.groupby(region_col)[profit_col].sum().sort_values(ascending=False)

    plt.figure(figsize=(10, 6))

    region_profit.plot(kind='bar')

    plt.title("Profit by Region")
    plt.xlabel("Region")
    plt.ylabel("Total Profit")

    plt.xticks(rotation=45)
    plt.show()
```




```
subcategory_col = None

for col in df.columns:
    if col.lower() in ['sub-category', 'subcategory', 'sub category']:
        subcategory_col = col
        break

if subcategory_col and sales_col:

    top_subcategories = (
        df.groupby(subcategory_col)[sales_col]
        .sum()
        .sort_values(ascending=False)
        .head(10)
    )

    plt.figure(figsize=(12, 6))

    top_subcategories.plot(kind='bar')

    plt.title("Top 10 Sub-Categories by Sales")
    plt.xlabel("Sub-Category")
    plt.ylabel("Sales")

    plt.xticks(rotation=45)
    plt.show()
```

```
if subcategory_col and profit_col:

    subcategory_profit = (
        df.groupby(subcategory_col)[profit_col]
        .sum()
        .sort_values()
    )

    print("Sub-categories with lowest profit:")
    display(subcategory_profit.head(10))
```

```
date_col = None

for col in df.columns:
    if 'date' in col.lower():
```

```
    if 'date' in col.lower():
        date_col = col
        break

if date_col and sales_col:

    df[date_col] = pd.to_datetime(df[date_col], errors='coerce')

    monthly_sales = (
        df.dropna(subset=[date_col])
        .set_index(date_col)[sales_col]
        .resample('M')
        .sum()
    )

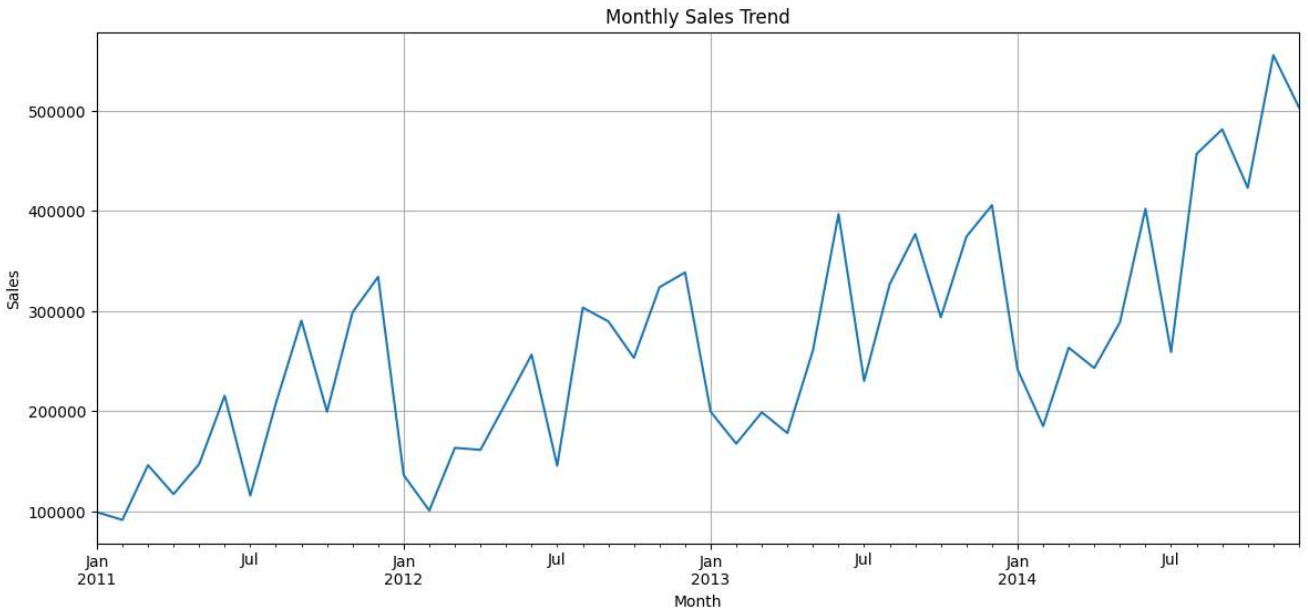
    plt.figure(figsize=(14, 6))

    monthly_sales.plot()

    plt.title("Monthly Sales Trend")
    plt.xlabel("Month")
    plt.ylabel("Sales")

    plt.grid(True)
    plt.show()
```

```
/tmp/ipykernel_2753/1354694228.py:15: FutureWarning: 'M' is deprecated and will be removed in a future version, please use 'ME'
  .resample('M')
```




```
if date_col and profit_col:

    monthly_profit = (
        df.dropna(subset=[date_col])
        .set_index(date_col)[profit_col]
        .resample('M')
        .sum()
    )

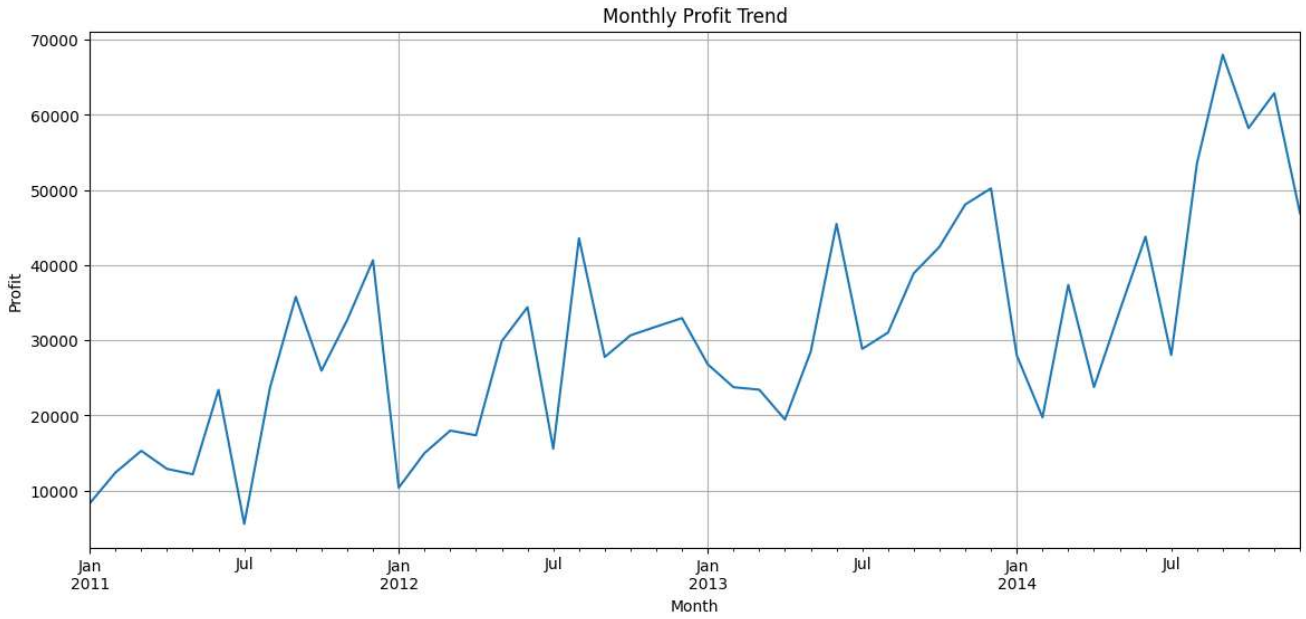
    plt.figure(figsize=(14, 6))

    monthly_profit.plot()

    plt.title("Monthly Profit Trend")
    plt.xlabel("Month")
    plt.ylabel("Profit")

    plt.grid(True)
    plt.show()
```

```
/tmp/ipykernel_2753/657958344.py:6: FutureWarning: 'M' is deprecated and will be removed in a future version, please use 'ME' in
  .resample('M')
```



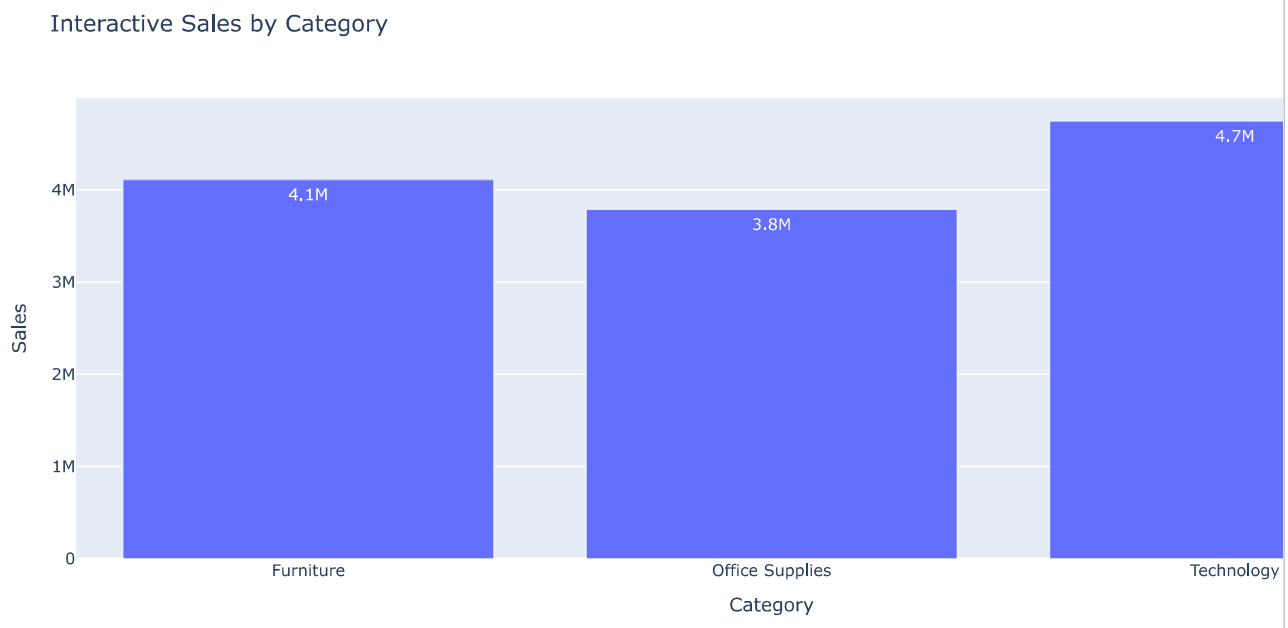

```
import plotly.express as px

if category_col and sales_col:

    category_data = (
        df.groupby(category_col, as_index=False)[sales_col]
        .sum()
    )

    fig = px.bar(
        category_data,
        x=category_col,
        y=sales_col,
        title="Interactive Sales by Category",
        text_auto='.2s'
    )

    fig.show()
```

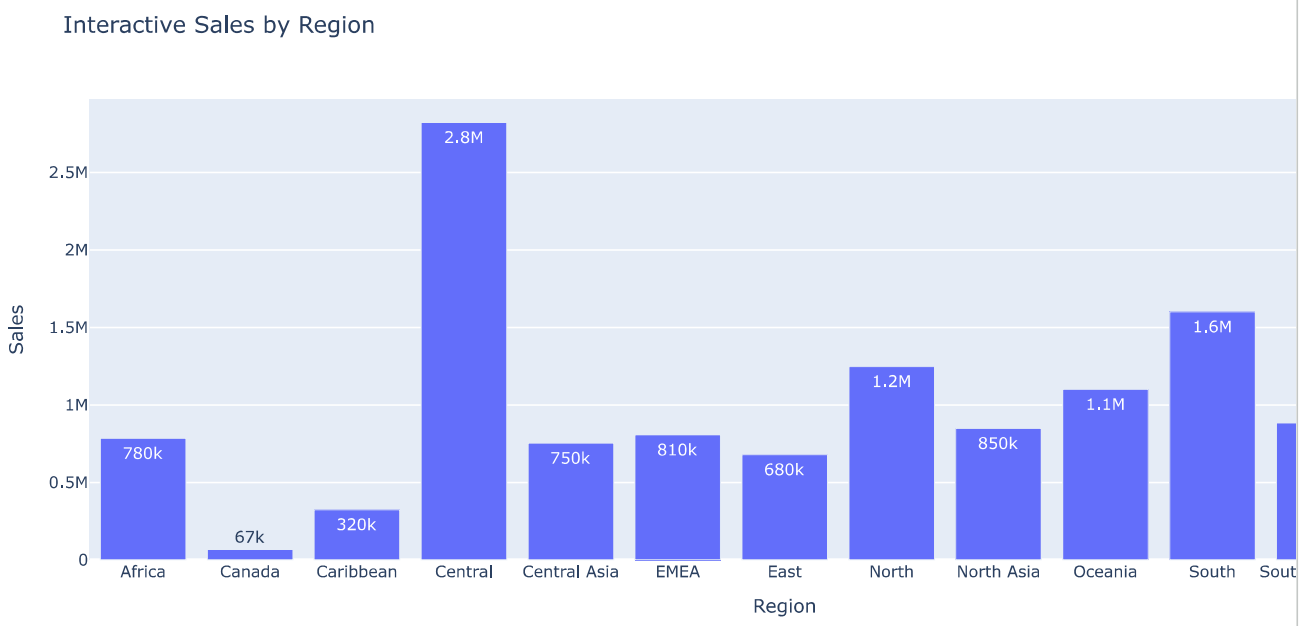



```
if region_col and sales_col:

    region_data = (
        df.groupby(region_col, as_index=False)[sales_col]
        .sum()
    )

    fig = px.bar(
        region_data,
        x=region_col,
        y=sales_col,
        title="Interactive Sales by Region",
        text_auto='.2s'
    )

    fig.show()
```



```
print("=" * 60)
print("              KEY BUSINESS INSIGHTS")
print("=" * 60)

if category_col and sales_col:
    best_category = (
        df.groupby(category_col)[sales_col]
        .sum()
        .idxmax()
    )

    best_category_sales = (
        df.groupby(category_col)[sales_col]
        .sum()
        .max()
    )

    print(f"\n1. Highest Sales Category: {best_category}")
    print(f"   Sales: {best_category_sales:,.2f}")

if category_col and profit_col:
    best_profit_category = (
        df.groupby(category_col)[profit_col]
        .sum()
        .idxmax()
    )

    print(f"\n2. Most Profitable Category: {best_profit_category}")

if region_col and sales_col:
    best_region = (
        df.groupby(region_col)[sales_col]
        .sum()
        .idxmax()
    )

    print(f"\n3. Highest Sales Region: {best_region}")

if region_col and profit_col:
    best_profit_region = (
        df.groupby(region_col)[profit_col]
        .sum()
        .idxmax()
    )

    print(f"\n4. Most Profitable Region: {best_profit_region}")

if subcategory_col and profit_col:
    worst_subcategory = (
        df.groupby(subcategory_col)[profit_col]
        .sum()
        .idxmin()
    )

    print(f"\n5. Lowest Profit Sub-Category: {worst_subcategory}")

print("\n6. Overall Profit Margin: {:.2f}%".format(profit_margin))

print("=" * 60)
```

```
============================================================
              KEY BUSINESS INSIGHTS
============================================================

1. Highest Sales Category: Technology
   Sales: 4,744,691.00

2. Most Profitable Category: Technology

3. Highest Sales Region: Central

4. Most Profitable Region: Central

6. Overall Profit Margin: 11.61%
============================================================
```

THEORY QUESTIONS AND ANSWERS

Q1. What is data storytelling? How is it different from data visualization?

Answer: Data storytelling is the process of combining data analysis, visualizations, and narrative to communicate meaningful insights to an audience. It explains not only what the data shows but also why the findings are important and what actions can be taken.

Data visualization mainly focuses on representing data using charts, graphs, maps, and other visual elements. Data storytelling goes one step further by combining visualizations with context, explanation, and recommendations.

Therefore, visualization presents the data, while storytelling explains the meaning and importance of the data.

Q2. Why is storytelling important in business analytics and decision-making?

Answer: Storytelling is important because it converts complex analytical results into a form that business stakeholders can understand easily. A good data story highlights important trends, problems, opportunities, and risks.

It helps decision-makers understand the significance of analytical findings and supports data-driven decisions. It also improves communication between data analysts and business stakeholders.